

Python Notes: *args, **kwargs, and Recursion

These notes explain *args, **kwargs, and recursion in detail, with examples, execution flow, comparisons, and quick revision points.

1. *args in Python

*args allows a function to accept any number of positional arguments.

Normal function:

```
def add(a, b):  
    return a + b
```

With *args:

```
def add(*args):  
    print(args)
```

Example: add(10, 20), add(10, 20, 30), or add(1, 2, 3, 4, 5) can all be used.

Inside the function, args is stored as a tuple. The * tells Python to pack multiple positional arguments into a tuple.

Example: Sum using *args

```
def calculate_sum(*args):  
    total = 0  
    for number in args:  
        total += number  
    return total
```

```
print(calculate_sum(10, 20))  
print(calculate_sum(10, 20, 30))  
print(calculate_sum(1, 2, 3, 4, 5))
```

Output:

```
30  
60  
15
```

Why is *args useful?

- When you do not know how many positional arguments will be passed.
- For creating flexible and reusable functions.
- When you want to process many values using a loop.

2. **kwargs in Python

****kwargs** allows a function to accept any number of keyword arguments.

Keyword arguments use the key=value form.

```
def student(**kwargs):  
    print(kwargs)
```

```
student(name="Aman", age=20, course="Python")
```

Output:

```
{'name': 'Aman', 'age': 20, 'course': 'Python'}
```

Inside the function, kwargs is stored as a dictionary. The ****** tells Python to pack keyword arguments into a dictionary.

Example: **kwargs

```
def student_details(**kwargs):  
    for key, value in kwargs.items():  
        print(key, ":", value)
```

```
student_details(name="Aman", age=20, course="Python", city="Lucknow")
```

Output:

```
name : Aman
```

```
age : 20
```

```
course : Python
```

```
city : Lucknow
```

3. Difference Between *args and **kwargs

Feature	*args	**kwargs
Arguments accepted	Positional	Keyword
Data type inside function	Tuple	Dictionary
Symbol	*	**
Example	func(10, 20)	func(a=10, b=20)
Purpose	Variable positional arguments	Variable keyword arguments

Easy way to remember:

*args → many positional arguments → tuple

**kwargs → many keyword arguments → dictionary

4. Using *args and **kwargs Together

```
def information(*args, **kwargs):  
    print("Arguments:", args)  
    print("Keyword arguments:", kwargs)
```

```
information(10, 20, 30, name="Aman", age=20)
```

Output:

Arguments: (10, 20, 30)

Keyword arguments: {'name': 'Aman', 'age': 20}

5. * and ** for Unpacking

The * and ** symbols can also be used when calling a function. This is called unpacking.

Sequence unpacking with *

```
numbers = [10, 20, 30]
```

```
def add(a, b, c):  
    return a + b + c
```

```
print(add(*numbers))
```

This is equivalent to add(10, 20, 30).

Dictionary unpacking with **

```
data = {"name": "Aman", "age": 20}
```

```
def student(name, age):  
    print(name)  
    print(age)
```

```
student(**data)
```

This is equivalent to student(name="Aman", age=20).

6. What is Recursion?

Recursion is a technique in which a function calls itself. A recursive function must have a stopping condition, called the base case.

Two important parts of recursion

1. Base Case — the condition that stops the recursive calls.
2. Recursive Case — the part where the function calls itself with a smaller or simpler problem.

Simple recursion example

```
def countdown(n):  
    if n == 0:  
        return  
    print(n)  
    countdown(n - 1)
```

```
countdown(5)
```

Output:

5

4
3
2
1

How it works

```
countdown(5)
  ↓
countdown(4)
  ↓
countdown(3)
  ↓
countdown(2)
  ↓
countdown(1)
  ↓
countdown(0) → stops
```

7. Recursion and the Call Stack

Each recursive function call is placed on the call stack. The stack keeps track of function calls that have not finished yet.

For `countdown(3)`, the stack grows roughly as:

```
countdown(3)
countdown(2)
countdown(1)
countdown(0)
```

When the base case returns, the previous calls can finish. This is why recursion uses stack memory.

8. Factorial Using Recursion

The factorial of n is the product of all positive integers from n down to 1.

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

Recursive definition:

$$n! = n \times (n - 1)!$$

$$0! = 1$$

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)
```

```
print(factorial(5))
```

Output: 120

Factorial step-by-step

```
factorial(5)
= 5 × factorial(4)
= 5 × 4 × factorial(3)
= 5 × 4 × 3 × factorial(2)
= 5 × 4 × 3 × 2 × factorial(1)
= 5 × 4 × 3 × 2 × 1 × factorial(0)
= 120
```

9. Sum of Numbers Using Recursion

```
def sum_n(n):
    if n == 0:
        return 0
    return n + sum_n(n - 1)
```

```
print(sum_n(5))
```

Output: 15

$\text{sum_n}(5) = 5 + \text{sum_n}(4) = 5 + 4 + \text{sum_n}(3) = \dots = 15$

10. Fibonacci Using Recursion

The Fibonacci sequence is:

0, 1, 1, 2, 3, 5, 8, 13, 21, ...

Each number is the sum of the previous two:

$F(n) = F(n - 1) + F(n - 2)$

Base cases:

$F(0) = 0$

$F(1) = 1$

```
def fibonacci(n):
    if n == 0:
        return 0
    if n == 1:
        return 1
    return fibonacci(n - 1) + fibonacci(n - 2)
```

```
print(fibonacci(6))
```

Output: 8

11. Recursion Example: Reverse a String

```
def reverse_string(s):  
    if len(s) == 0:  
        return ""  
    return reverse_string(s[1:]) + s[0]
```

```
print(reverse_string("hello"))
```

Output: olleh

12. Recursion Example: Sum of a List

```
def list_sum(numbers):  
    if len(numbers) == 0:  
        return 0  
    return numbers[0] + list_sum(numbers[1:])
```

```
numbers = [10, 20, 30, 40]
```

```
print(list_sum(numbers))
```

Output: 100

13. What Happens Without a Base Case?

If a recursive function never reaches a stopping condition, it keeps calling itself until Python reaches its recursion limit.

```
def test(n):  
    print(n)  
    test(n - 1)
```

```
test(5)
```

Eventually Python raises:

RecursionError: maximum recursion depth exceeded

14. Recursion vs Iteration

Recursion	Iteration	General idea
Function calls itself	Uses loops	Different control-flow approaches
Uses call stack	Usually uses less stack memory	Memory consideration
Can make recursive problems easier to express	Often simpler for repetition	Readability
Can cause RecursionError	Usually avoids recursion-depth limits	Limitation

15. When Should You Use Recursion?

Recursion is useful when a problem naturally breaks into smaller versions of itself.

- Factorial
- Fibonacci
- Tree traversal
- Graph algorithms
- Binary search
- Divide-and-conquer algorithms
- Directory/file traversal
- Backtracking
- Merge sort and quick sort

For simple repetition, a loop is often easier and more memory-efficient.

16. One-Page Exam Revision

`*args`

→ Variable positional arguments

→ Stored as tuple

`**kwargs`

→ Variable keyword arguments

→ Stored as dictionary

Recursion

→ Function calls itself

→ Needs a base case

→ Has a recursive case

→ Uses the call stack

17. Easy Memory Tricks

`*args` → many positional arguments → tuple

`**kwargs` → many keyword arguments → dictionary

Recursion → function calling itself → must eventually reach a base case

End of Notes